SC4015 Cyber Physical Systems Security

Group Project Report

Team no 1
Lab Group: TUT_SCEL, LAB-TEL2 & 4

Name	Matriculation Number
Krishna Nageswaran Jeevika	U2223941D
Nithin Raj Murali Babu	U2223360E
Sushmita Ramaratnam	U2222958B

Contents

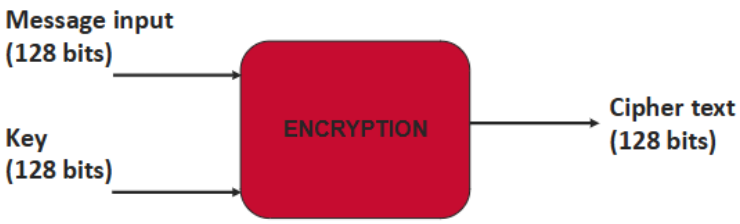
Team members 1
1. Introduction 3
1.1 Advanced Encryption Standard (AES) 3
2. Experimental Setup 5
Results 8
Conclusion 20

1. Introduction

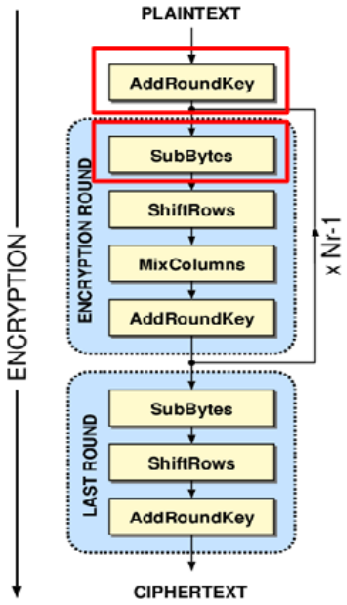
Given a set of power traces in the waveform.csv file, we are required to implement Correlation Power Analysis (CPA) on these traces to extract all 16 bytes of the secret key.

1.1 Advanced Encryption Standard (AES)

The AES crypto algorithm is a block cipher. This is a type of symmetric-key cryptographic algorithm that utilises the same key to perform both encryption and decryption of data. The AES algorithm is widely applied in a range of domains due to its robustness, and it would take even a supercomputer a billion years to break this algorithm through brute force attack.



Structure of AES algorithm



Here, the input is a plaintext of 128 bits, which are organized into blocks of 16 bytes. AES is an amalgamation of various functions, including AddRoundKey, 9 rounds of SubBytes, ShiftRows, MixColumns, and AddRoundKey, and a final round of SubByte, ShiftRows, and AddRoundKey.

AddRoundKey
This is the first operation where 16 bytes of plaintext are XOR'ed with 16 bytes of the secret key. The output obtained from AddRoundKey is 16 bytes of the intermediate state.

SubBytes

This is the second operation, which involves substituting each byte in the output of AddRoundKey with another byte. This substitution is performed based on the AES S-Box, a lookup table containing 256 values. Here, for every value, there is a corresponding lookup value. The SBOX operation is a memory-based lookup where input values are mapped to corresponding outputs.

1.2 Correlation Power Analysis (CPA)

The structure of the AES algorithm makes it infeasible to break through brute force. However, its hardware implementation introduces a vulnerability in the form of Power Analysis Side-Channel Attacks. In particular, operations such as the SBOX substitution involve data movement between registers and memory, where wires toggle between logic states 0 and 1. These transitions cause charging and discharging of wires, resulting in variations in power consumption. This behaviour can be explained using leakage models. When the device is idle, data lines are typically precharged to logic 1. During computation, some of these lines switch to logic 0 depending on the data being processed. As a result, the power consumption of the device depends on the number of bits that change state.

This relationship is commonly modelled using the Hamming weight model, which estimates power consumption based on the number of '1' bits present in a value.

In the context of the attack, the output of the SBOX operation serves as a critical intermediate value that influences these bit transitions. By modelling the power consumption corresponding to different possible SBOX outputs and comparing it with the observed power traces,

Correlation Power Analysis (CPA) can be used to statistically recover the secret key of the MCU.

1.3 Waveform.csv

We have been provided with a CSV file (waveform.csv) which contains 110 power traces, each corresponding to a separate encryption. The first column represents the plaintext input, while the second column is the corresponding ciphertext output. The remaining 2500 columns store the power trace samples with floating-point values ranging from 0 to 1, which are captured each time the MCU performs an AES encryption on the given plaintext. Plotting a row reveals the power consumption for a single encryption, and using 110 samples, along with plaintext and ciphertext (16 bytes each), we can recover the 16-byte secret key.

2. Experimental Setup

The Correlation Power Analysis experiment has been carried out in a Python notebook through Google Colab.

2.1 Setting up the SBOX

```python

# AES S-Box in the form of a matrix 'SBOX'
SBOX = np.array([
    0x63, 0x7C, 0x77, 0x7B, 0xF2, 0x6B, 0x6F, 0xC5, 0x30, 0x01, 0x67, 0x2B, 0xFE, 0xD7, 0xAB, 0x76,
    0xCA, 0x82, 0xC9, 0x7D, 0xFA, 0x59, 0x47, 0xF0, 0xAD, 0xD4, 0xA2, 0xAF, 0x9C, 0xA4, 0x72, 0xC0,
    0xB7, 0xFD, 0x93, 0x26, 0x36, 0x3F, 0xF7, 0xCC, 0x34, 0xA5, 0xE5, 0xF1, 0x71, 0xD8, 0x31, 0x15,
    0x04, 0xC7, 0x23, 0xC3, 0x18, 0x96, 0x05, 0x9A, 0x07, 0x12, 0x80, 0xE2, 0xEB, 0x27, 0xB2, 0x75,
    0x09, 0x83, 0x2C, 0x1A, 0x1B, 0x6E, 0x5A, 0xA0, 0x52, 0x3B, 0xD6, 0xB3, 0x29, 0xE3, 0x2F, 0x84,
    0x53, 0xD1, 0x00, 0xED, 0x20, 0xFC, 0xB1, 0x5B, 0x6A, 0xCB, 0xBE, 0x39, 0x4A, 0x4C, 0x58, 0xCF,
    0xD0, 0xEF, 0xAA, 0xFB, 0x43, 0x4D, 0x33, 0x85, 0x45, 0xF9, 0x02, 0x7F, 0x50, 0x3C, 0x9F, 0xA8,
    0x51, 0xA3, 0x40, 0x8F, 0x92, 0x9D, 0x38, 0xF5, 0xBC, 0xB6, 0xDA, 0x21, 0x10, 0xFF, 0xF3, 0xD2,
    0xCD, 0x0C, 0x13, 0xEC, 0x5F, 0x97, 0x44, 0x17, 0xC4, 0xA7, 0x7E, 0x3D, 0x64, 0x5D, 0x19, 0x73,
    0x60, 0x81, 0x4F, 0xDC, 0x22, 0x2A, 0x90, 0x88, 0x46, 0xEE, 0xB8, 0x14, 0xDE, 0x5E, 0x0B, 0xDB,
    0xE0, 0x32, 0x3A, 0x0A, 0x49, 0x06, 0x24, 0x5C, 0xC2, 0xD3, 0xAC, 0x62, 0x91, 0x95, 0xE4, 0x79,
    0xE7, 0xC8, 0x37, 0x6D, 0x8D, 0xD5, 0x4E, 0xA9, 0x6C, 0x56, 0xF4, 0xEA, 0x65, 0x7A, 0xAE, 0x08,
    0xBA, 0x78, 0x25, 0x2E, 0x1C, 0xA6, 0xB4, 0xC6, 0xE8, 0xDD, 0x74, 0x1F, 0x4B, 0xBD, 0x8B, 0x8A,
    0x70, 0x3E, 0xB5, 0x66, 0x48, 0x03, 0xF6, 0x0E, 0x61, 0x35, 0x57, 0xB9, 0x86, 0xC1, 0x1D, 0x9E,
    0xE1, 0xF8, 0x98, 0x11, 0x69, 0xD9, 0x8E, 0x94, 0x9B, 0x1E, 0x87, 0xE9, 0xCE, 0x55, 0x28, 0xDF,
    0x8C, 0xA1, 0x89, 0x0D, 0xBF, 0xE6, 0x42, 0x68, 0x41, 0x99, 0x2D, 0x0F, 0xB0, 0x54, 0xBB, 0x16,
], dtype=np.uint8)

```

The 256 values present in the AES S-Box have been represented in the form of a matrix stored in the variable ‘SBOX’.

2.2 Hamming Weight Lookup Table

```python

# Hamming Weight
HW_TABLE = np.array([bin(x).count("1") for x in range(256)], dtype=np.uint8)

```

A Hamming Weight Lookup Table is created and stored in the variable HW_TABLE for all possible 8-bit values (ranging from 0 to 255). The Hamming weight was calculated by converting the decimal values into binary and computing the number of 1’s.

2.3 Power Model Construction

```python

for b in range(16):
    n_traces = traces.shape[0]
    hyp_hw = np.zeros((256, n_traces))
    for k in range(256):
        hyp_hw[k, :] = HW_TABLE[SBOX[pt_matrix[:, b] ^ k]]

```

At each key byte position, 256 possible key values are guessed and each key candidate is XOR'ed with the corresponding plaintext byte. The resulting values are passed through the S-Box and converted into their Hamming weight. The computed values are recorded in hyp_hw[k, :] as the predicted power for each key guess.

2.4 Correlation Computation

```python
    model_c = hyp_hw - hyp_hw.mean(axis=1, keepdims=True)
    trace_c = traces - traces.mean(axis=0, keepdims=True)

    num = np.dot(model_c, trace_c)
    den = np.sqrt(np.sum(model_c**2, axis=1, keepdims=True)) * np.sqrt(np.sum(trace_c**2, axis=0, keepdims=True))

    corr = np.abs(num / (den + 1e-10))    # corr is the 2D matrix => 256 key candidates * time samples
```

Pearson correlation coefficient is utilised to compare the measured and predicted power traces. This enables us to identify how closely key guesses align with the observed power patterns.

2.5 Maximum Correlation Computation and Key Byte Recovery

```python

    max_corrs = np.max(corr, axis=1)
    best_guess = np.argmax(max_corrs)

    recovered_key.append(best_guess)
    key_strengths.append(max_corrs[best_guess])

```

For each key guess, the maximum correlation value is calculated. The candidate key having the highest correlation score is chosen as the recovered key byte.

2.6 Visualisation

```python
    #fig.suptitle(f"AES Analysis: Byte {b}", fontsize=14, fontweight='bold')
    fig.suptitle(
    f"AES CPA: Byte {b}\nRunning Key: {running_key_display}",
    fontsize=12,
    fontweight='bold'
    )
    ax1.plot(max_corrs, color='darkgray', alpha=0.6)


    ax1.plot(best_guess, max_corrs[best_guess], 'ro', markersize=4)
    ax1.scatter(best_guess, max_corrs[best_guess],
                facecolors='none', edgecolors='red',
                s=150, linewidths=2)

    ax1.set_title(f"Best Key Guess = {best_guess_hex} (Decimal: {best_guess})")

    ax1.set_xlabel("Key Candidates")
    ax1.set_ylabel("Correlation")
    ax1.grid(True, axis='y', linestyle=':', alpha=0.5)

```

The correlation values of all candidate keys are plotted. And the correct or closest guess is highlighted with a red circle.

2.7 Recovered (Secret) Key

```python
# Final Output (Secret Key)
final_key_hex = "".join([f"{k:02x}" for k in recovered_key])
print("\nRECOVERED KEY: " + " " + " ".join(f"{k:02X}" for k in recovered_key))

```

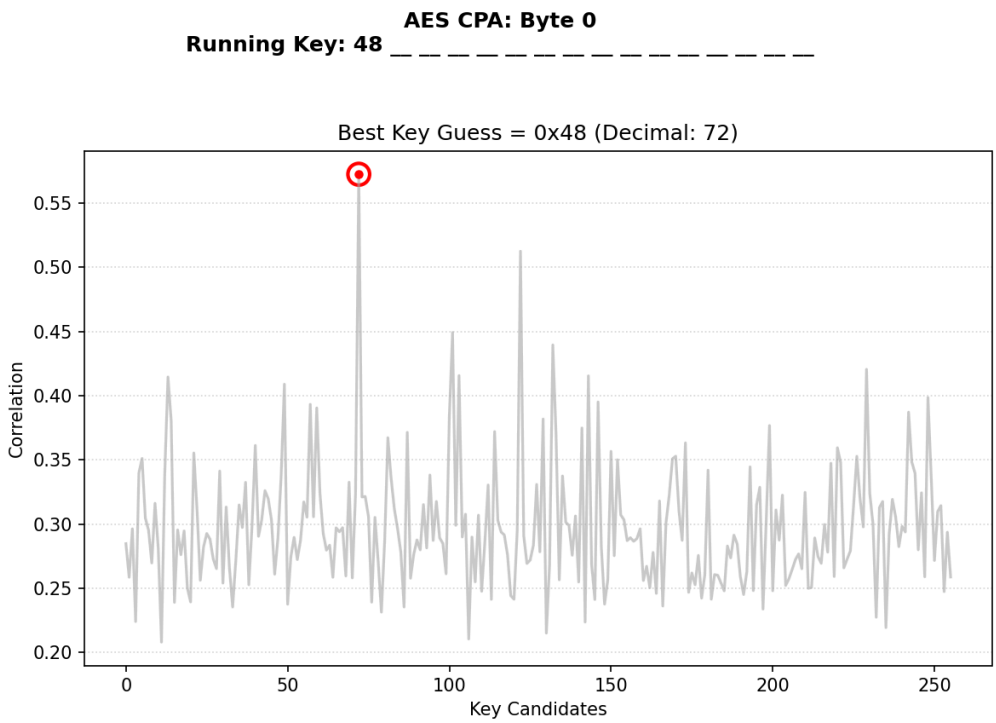
Finally, the recovered key bytes are combined and printed out in the hexadecimal format, forming the complete AES secret key.

```
RECOVERED KEY:  48 9D B4 B3 F3 17 29 61 CC 2B CB 4E D2 E2 8E B7
```

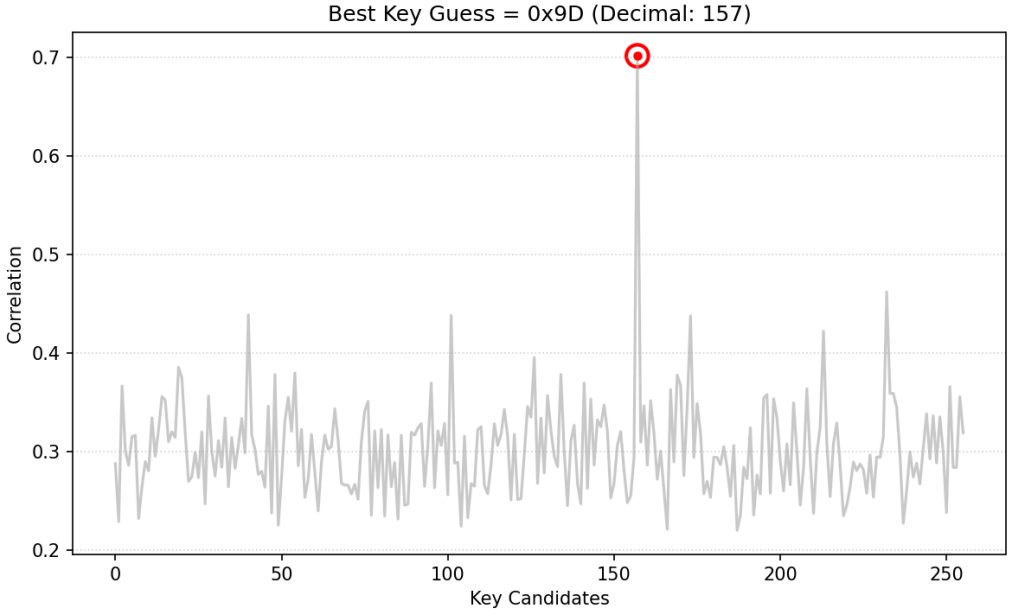
3. Results

Plots

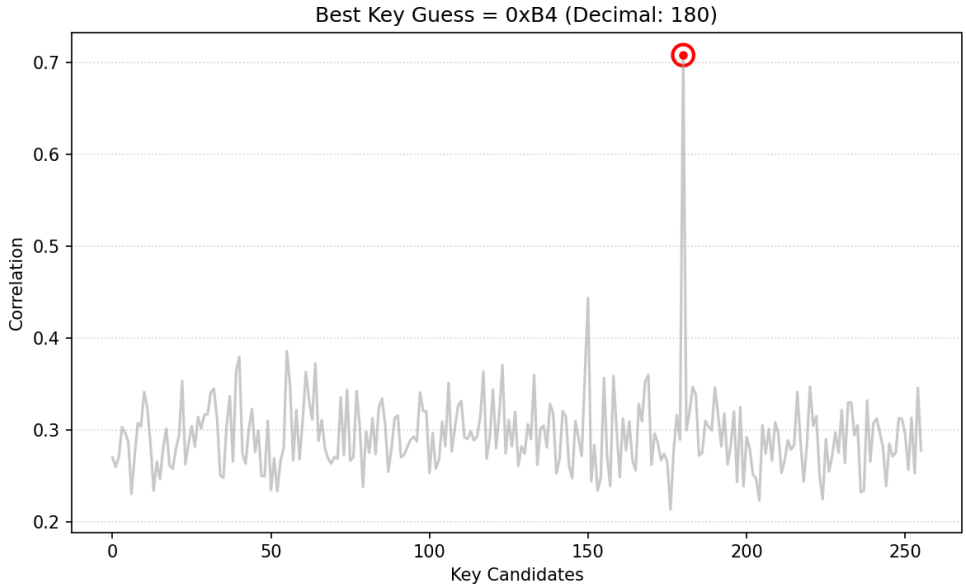
As stated previously, 16 correlation plots were obtained. Within each plot, the correlation coefficients for all 256 key candidates are calculated and displayed accordingly. The x-axis here is the value of the key candidate, ranging from 0 to 255. The y-axis is the corresponding computed correlation coefficient for each key candidate. In each plot, there is a red circle to show the correct key byte that corresponds to the maximum correlation value.



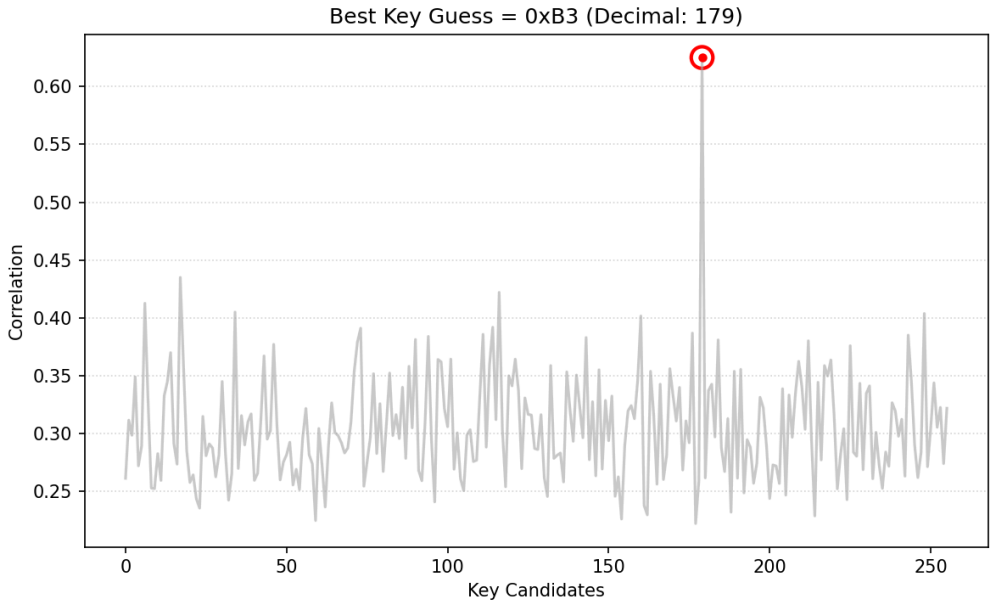
AES CPA: Byte 1
Running Key: 48 9D __ __ __ __ __ __ __ __ __ __ __ __ __ __



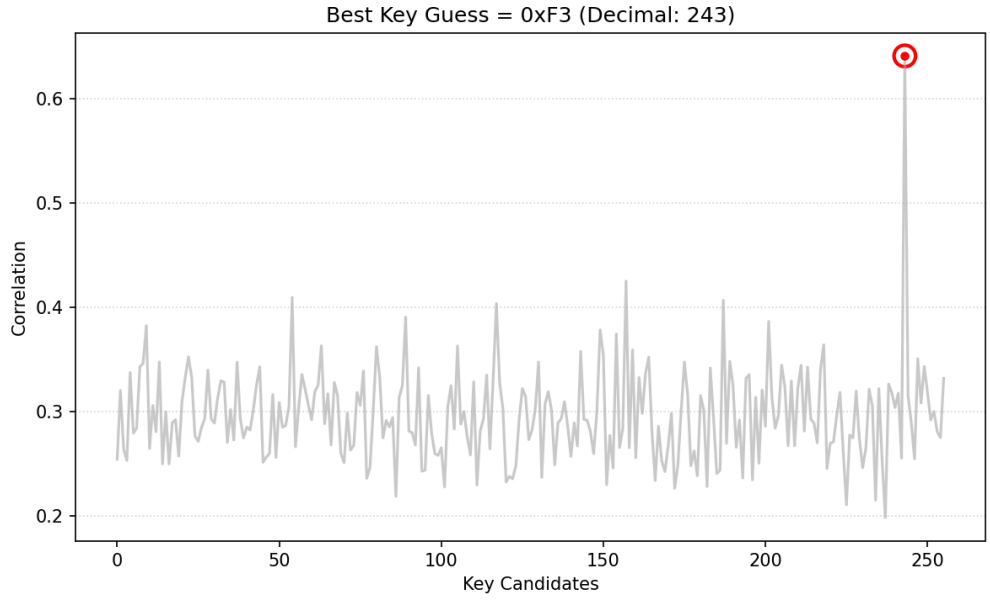
AES CPA: Byte 2
Running Key: 48 9D B4 __ __ __ __ __ __ __ __ __ __ __ __ __



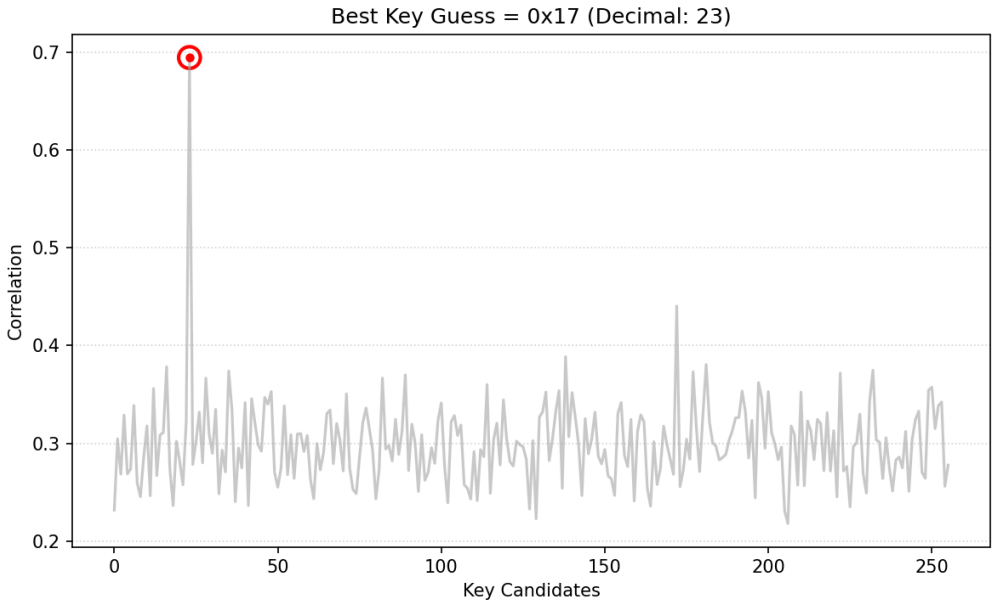
AES CPA: Byte 3
Running Key: 48 9D B4 B3 __ __ __ __ __ __ __ __ __ __ __ __



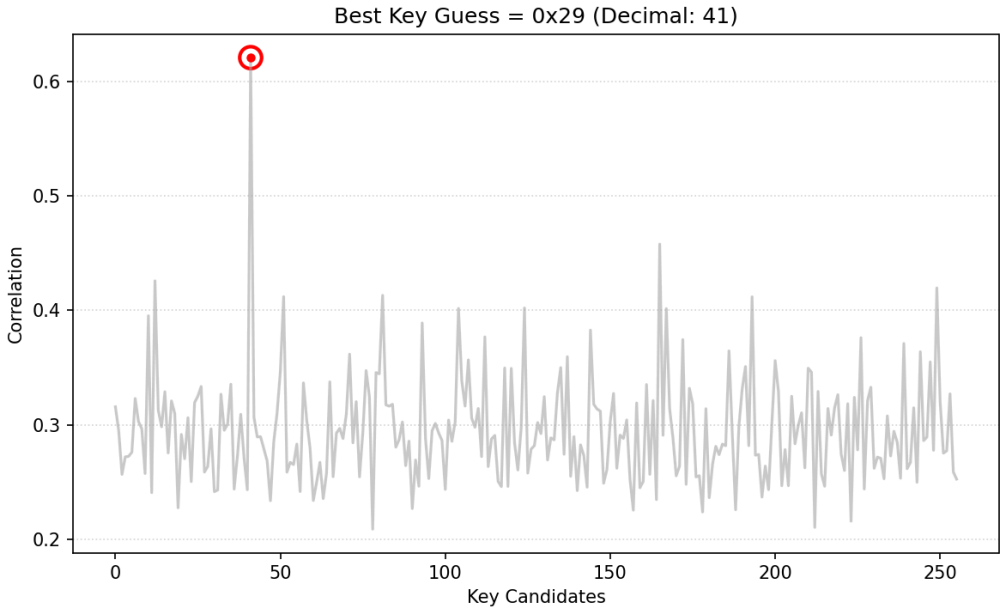
AES CPA: Byte 4
Running Key: 48 9D B4 B3 F3 __ __ __ __ __ __ __ __ __ __ __



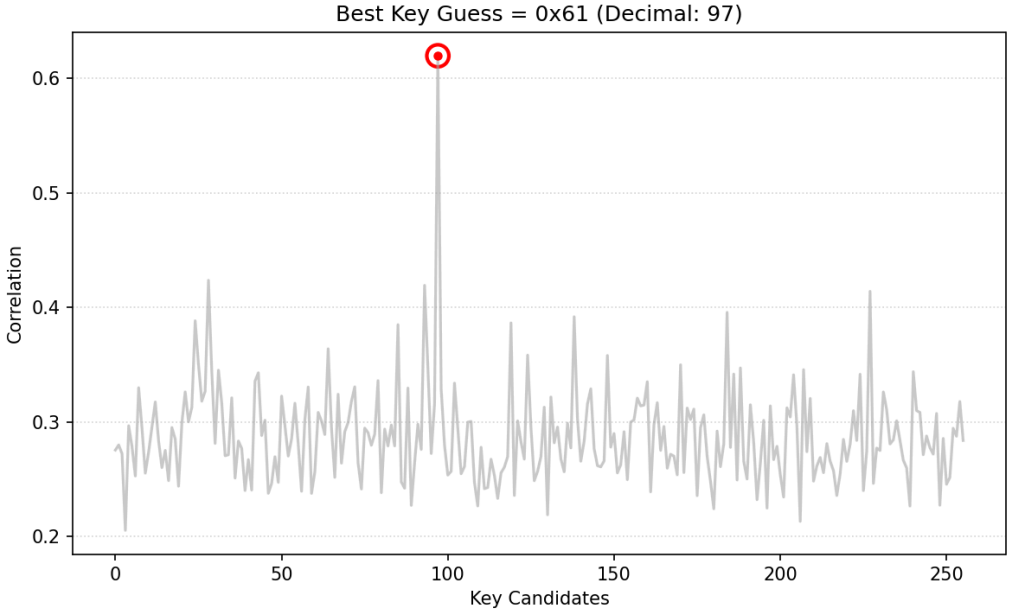
AES CPA: Byte 5
Running Key: 48 9D B4 B3 F3 17 __ __ __ __ __ __ __ __ __ __



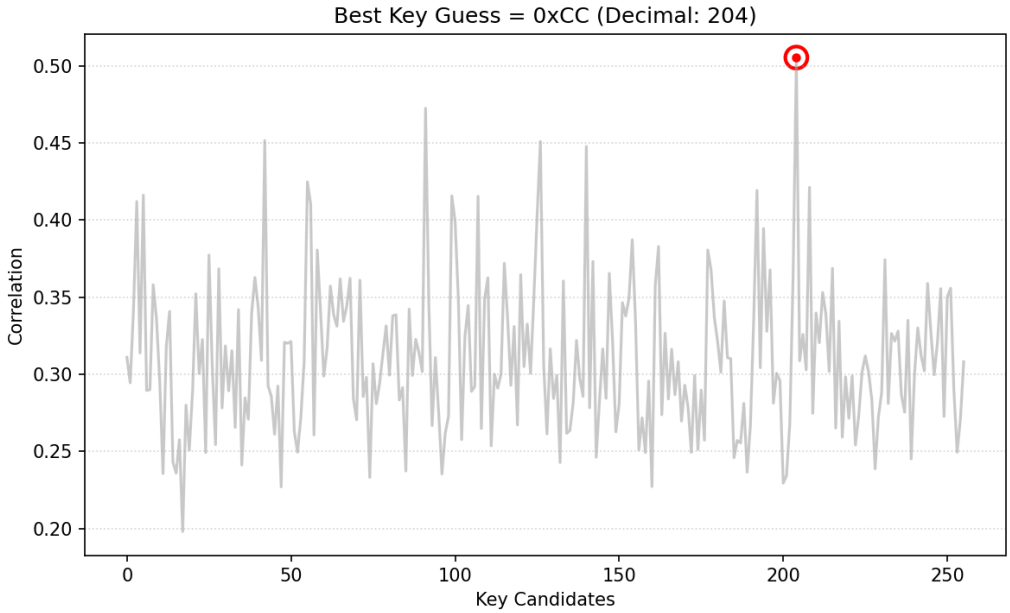
AES CPA: Byte 6
Running Key: 48 9D B4 B3 F3 17 29 __ __ __ __ __ __ __ __ __



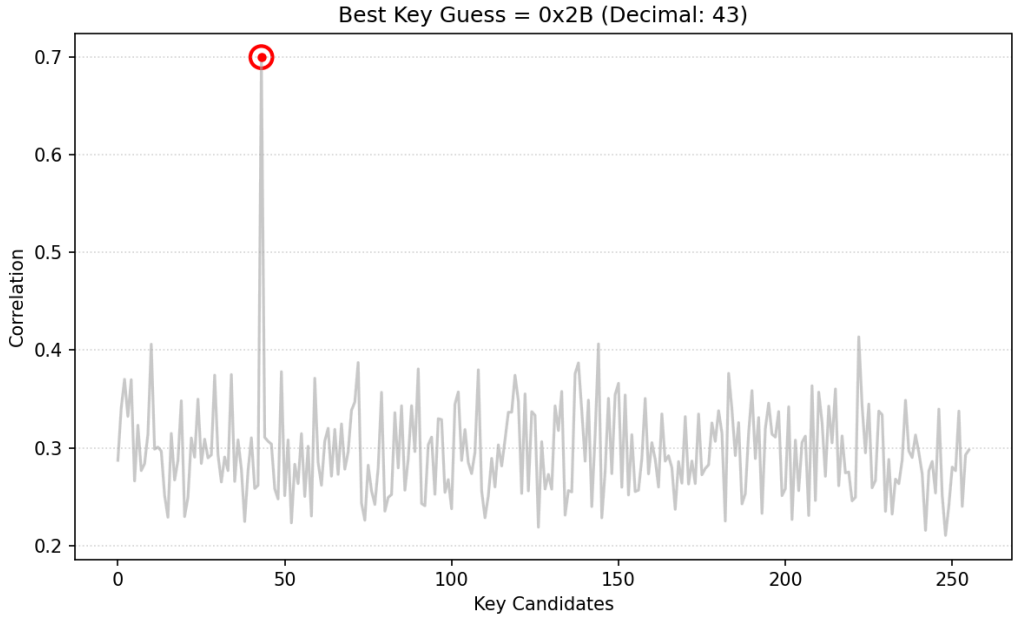
AES CPA: Byte 7
Running Key: 48 9D B4 B3 F3 17 29 61 __ __ __ __ __ __ __ __



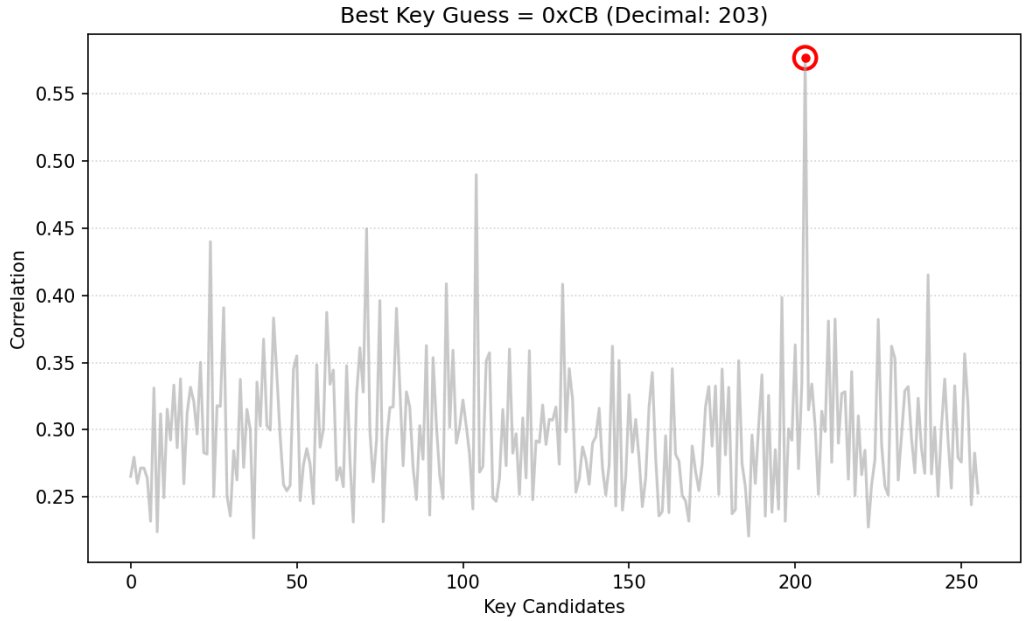
AES CPA: Byte 8
Running Key: 48 9D B4 B3 F3 17 29 61 CC __ __ __ __ __ __ __



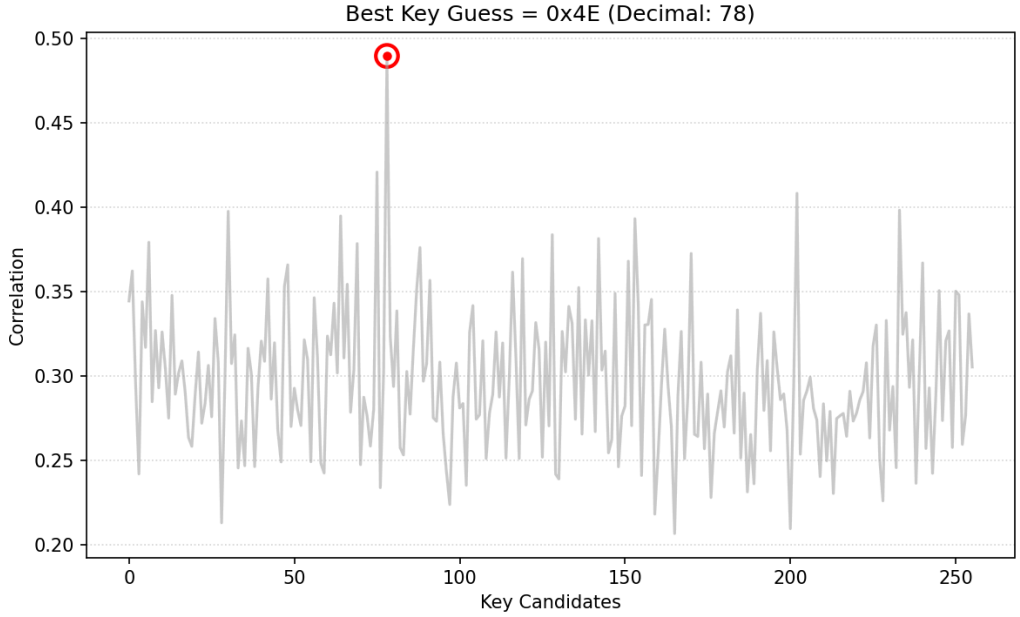
AES CPA: Byte 9
Running Key: 48 9D B4 B3 F3 17 29 61 CC 2B __ __ __ __ __ __



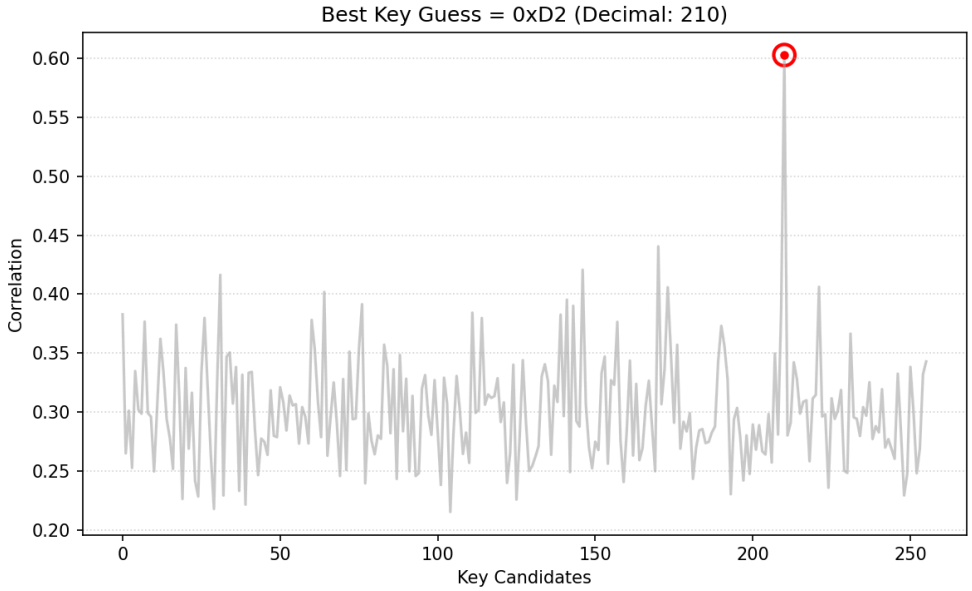
AES CPA: Byte 10
Running Key: 48 9D B4 B3 F3 17 29 61 CC 2B CB __ __ __ __ __



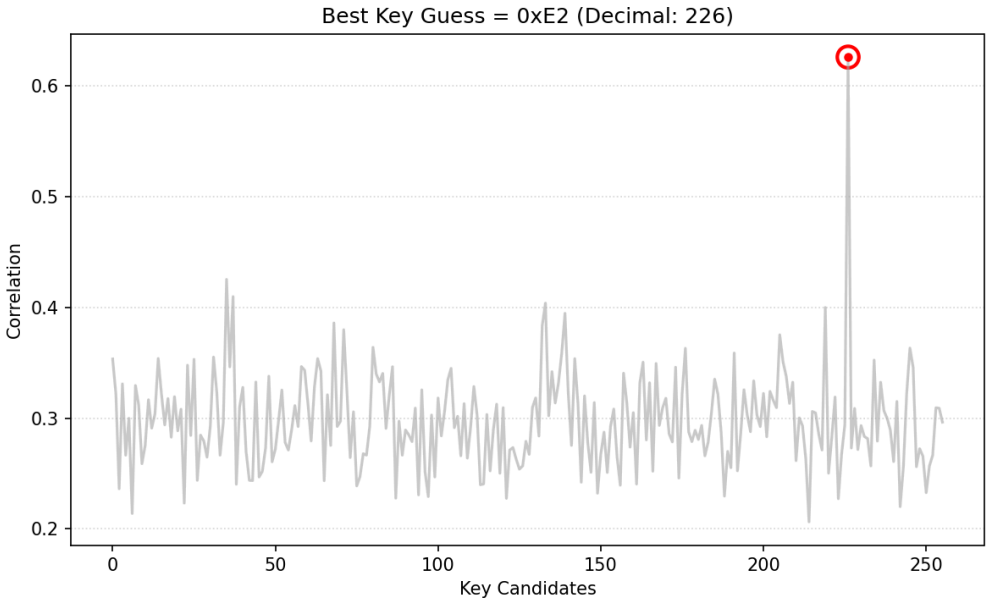
AES CPA: Byte 11
Running Key: 48 9D B4 B3 F3 17 29 61 CC 2B CB 4E __ __ __ __



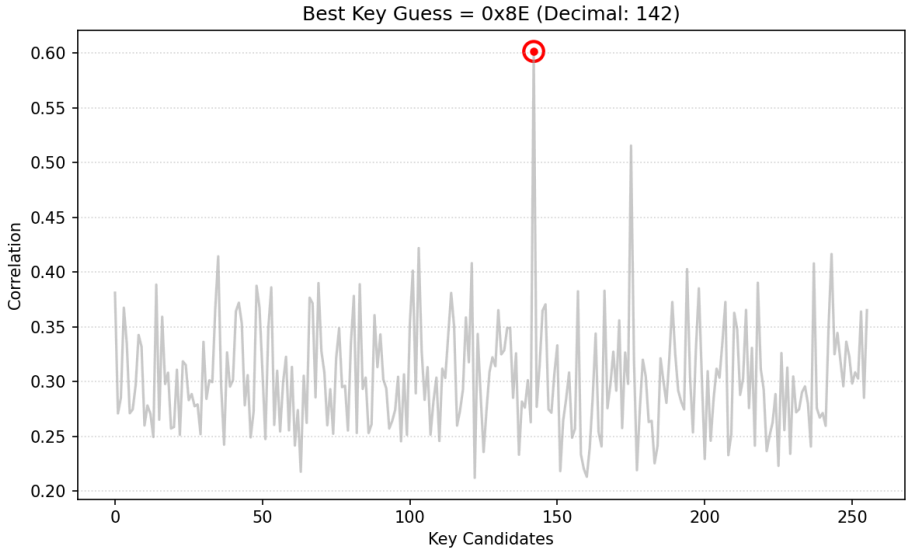
AES CPA: Byte 12
Running Key: 48 9D B4 B3 F3 17 29 61 CC 2B CB 4E D2 __ __ __

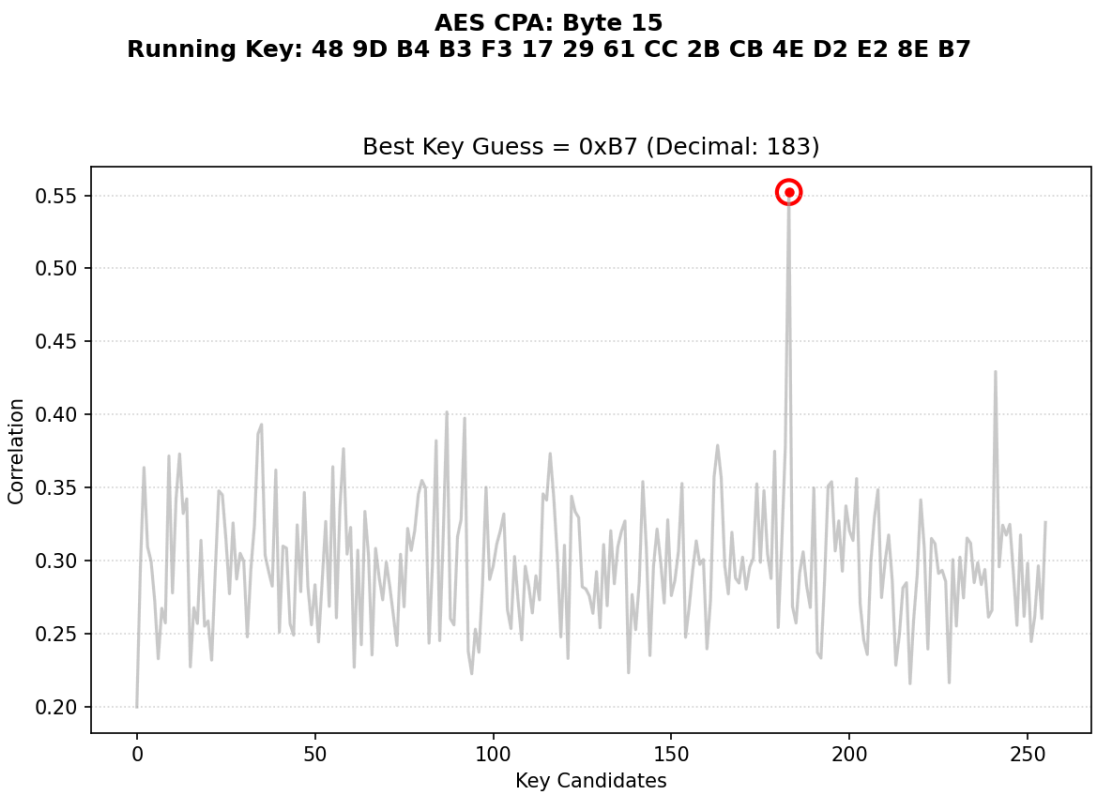


AES CPA: Byte 13
Running Key: 48 9D B4 B3 F3 17 29 61 CC 2B CB 4E D2 E2 __ __



AES CPA: Byte 14
Running Key: 48 9D B4 B3 F3 17 29 61 CC 2B CB 4E D2 E2 8E __





4. Conclusion

From our findings, the recovered key is:
`48 9D B4 B3 F3 17 29 61 CC 2B CB 4E D2 E2 8E B7`

Through this experiment, we can conclude that Correlation Power Analysis enables extraction of the secret AES key by using information leaked through power consumption during device operation. One just has to compare the correlation data and the corresponding plots.

Additionally, the experiment also shows us the dangers of implementing cryptographic algorithms without proper protection against side-channel attacks. Robust countermeasures in the form of both hardware and software are required in real-world applications and systems.